09 — Signature & stamp verification

What it does: three separate, deliberately modest things.

1. **Detection** (automatic, every PDF): locates where a handwritten signature or stamp/seal *appears* and whether a document of this kind would normally carry one. Presence and placement only.
2. **Comparison** (on demand): after a reviewer draws a *reference* signature, compares it to the same kind of region on every other document in the case — a qualitative vision-model opinion.
3. **Reuse check** (on demand, no model): notices when the very same signature *image* appears under two documents — which a real hand never produces.

None of these verifies who signed. They give a human reviewer something to look at.

Code: `services/signature_detection_service.py`, `services/signature_comparison_service.py`, `tasks/signature_detection_task.py`, `tasks/signature_comparison_task.py`, `api/signatures.py`, `models/signature_reference.py`, `models/signature_match.py`, frontend `components/upload/SignatureReferenceCreator.tsx`.

1. Detection

`run_signature_detection` (PDF only) renders the pages and calls `detect_signatures(llm, pages, pdf_bytes)`, analysing at most the first **10** pages. For each page it sends a copy of the render with a light **labelled 0.1 grid** overlay — vision models are poor at estimating "where" as a fraction of a page (observed off by up to ~0.3), so the grid lets them read coordinates off the image — and receives a `PageSignatureDetection`:

```
class PageSignatureDetection:
    signature_expected: bool                    # would this kind of document normally carry one?
    regions: list[DetectedSignatureRegion]      # kind: "signature"|"stamp", description,
                                                # confidence: low|medium|high, bounding_box
```

Box refinement (accuracy fix)

The model's box is only a first guess, so each region is refined before it is stored:

1. **`embedded_image`** — if the PDF has an embedded raster image near the model's box (the usual case for a pasted or scanned signature/stamp), the image's exact placement rectangle replaces the guess. Boxes and images are paired nearest-first, one-to-one, so a signature and a stamp never claim the same image. Page-sized images (> 35 % of the page) and rotated pages are ignored.
2. **`second_look`** — otherwise the model is shown a zoomed window around its own box (grown by 10 % of the page width and 8 % of its height on each side) and asked again; the answer is mapped back to page coordinates. Skipped if the window is blank.
3. **`none`** — neither worked; the (padded) first-pass box is kept.

Refined boxes are trimmed to the ink inside them (+0.4 % margin). The chosen method is recorded per region in `refinement`. Without this, boxes were seen to land on the printed name under a signature instead of the signature itself.

Result (stored as a `document_checks` row, `check_type = signature_stamp_detection`)

```
{
  "result": "pass",                       // "flag" only when a signature was expected but none found
  "details": {
    "signature_expected": true,
    "detected": [
      {"kind": "signature", "confidence": "high", "refinement": "embedded_image",
       "description": "Handwritten signature in blue ink under the label …",
       "bounding_box": {"page": 1, "x": 0.130, "y": 0.493, "width": 0.177, "height": 0.053}},
      {"kind": "stamp", "confidence": "high", "refinement": "embedded_image", "description": "Round company
stamp …",
       "bounding_box": {"page": 1, "x": 0.441, "y": 0.485, "width": 0.118, "height": 0.087}}
    ],
    "bounding_box": {"page": 1, "x": 0.130, "y": 0.493, "width": 0.177, "height": 0.053}   // the "primary"
region, or null
  }
}
```

`bounding_box` is the primary region: highest confidence, signature preferred on a tie. `detected` keeps every region with its `kind`; results stored before that field existed have only `bounding_box`. Presence alone is a **pass**.

2. Reference creation

`POST /cases/{case_id}/documents/{document_id}/signature-references` with a `SignatureReferenceCreate`:

```
{"person_name": "Sultan Al-Dhaheri",       // typed by the uploader/reviewer; NEVER auto-filled
 "bounding_box": {"page": 1, "x": 0.10, "y": 0.49, "width": 0.22, "height": 0.05},
 "is_library": true}                        // default true
```

Allowed for the case's owner (who sets the reference while uploading) and any reviewer of the company; anyone else gets a 404, and platform admins are read-only. The endpoint renders the PDF page, crops the normalized box, uploads a PNG to `companies/{company_id}/cases/{case_id}/signatures/{crop_id}.png`, inserts the `signature_references` row and enqueues `run_signature_comparison` (`vision_queue`). If the crop fails (render error) the row is still created with a null image URL and comparisons record `cannot_determine`. The box is drawn in the UI on a `react-pdf` viewer (`SignatureReferenceCreator`), offered after upload. References, like everything else, belong to one company: the library flag never makes a reference visible to another company.

3. Comparison (`run_signature_comparison`)

For the new reference, the task:

1. **Infers the reference's kind** (`signature` or `stamp`) by overlapping its box with the source document's detected regions (IoU > 0.1). Unknown if nothing overlaps.
2. For each **other** document in the case, picks the detected region of the **same kind** (a signature is never compared with a stamp); documents with no region of that kind are skipped. If the kind is unknown, the document's primary region is used. Documents whose detection has not finished are skipped and picked up when it does (the detection task re-triggers this one; it is idempotent and skips already-compared pairs).
3. Crops the target region (fresh blob name per attempt).
4. **Reuse check** (signature references only): see below. A match is stored and the model is **not** called.
5. Otherwise calls `LLMService.compare_signatures(ref_image, target_image)` — a strict-JSON vision call at `temperature = 0` returning `{reasoning, result}` (`reasoning` first, so the model commits its thinking before naming a verdict). Advisory wording only.

6. Stores a `signature_matches` row, audits `signature_comparison_completed` `{comparisons_attempted, comparisons_stored, reference_kind, identical_reuse_found}`, and re-scores the case.

If the vision model is not configured, non-reuse targets get `cannot_determine`; the reuse check still runs because it needs no model.

The reuse check (classical CV, no model)

A model asked "do these look alike?" calls two identical images "consistent" — reassurance, when the real signal is the opposite: nobody signs pixel-for-pixel the same twice, so an exact match means the same image file was inserted into both documents. That can be legitimate (a stored e-signature image), so on its own it is a modest flag.

`find_signature_reuse` trims both crops to their ink, requires similar proportions (± 6 %), resamples to 256 px wide, blurs, and takes the **maximum normalized cross-correlation** allowing ±4 px of shift. Measured on real files: identical copies score 0.999, a 60 % rescale 0.97, JPEG recompression 0.999, a wobbled re-signing 0.40, a different signature 0.17. The threshold is **0.93**.

On a match it reads the **printed line beneath each signature** from the PDF's own text layer (`printed_label_below`, the first text line within 7 % of page height below the box) and compares them with `rapidfuzz.fuzz.token_set_ratio` (< 60 = different people):

Verdict	Meaning
`reused_different_signer`	identical image **and** the printed signer text differs
`identical_reuse`	identical image; printed text similar or unreadable (e.g. a scan)

It never runs for **stamp** references — a company's rubber stamp is legitimately identical every time.

Output shape

Verdicts (`signature_match_result` enum) and their UI labels:

`result`	Source	Label
`consistent`	vision model	Visually consistent with reference on file
`possibly_consistent`	vision model	Possibly consistent with reference on file
`inconsistent`	vision model	Visual appearance inconsistent with reference
`cannot_determine`	vision model / failure	Cannot determine — insufficient image quality
`identical_reuse`	pixel check	Pixel-identical to reference — same image appears to be reused
`reused_different_signer`	pixel check	Pixel-identical to reference, but printed signer differs

`SignatureMatchResponse` (`schemas/signature.py`):


```
{"id": "uuid", "document_id": "uuid", "case_id": "uuid", "signature_reference_id": "uuid",
 "reference_person_name": "Sultan Al-Dhaheri", "comparison_scope": "in_case",
 "result": "reused_different_signer", "result_label": "Pixel-identical to reference, but printed signer
differs",
 "reasoning": "The signature here is pixel-for-pixel identical to the reference … reference: 'م . سلطان
الظاهري'; this document: 'سعيد المنصوري' …",
 "compared_at": "2026-09-20T23:10:15Z"}
```


There is **no numeric score** on the model verdicts, by design.

Known limitations

- **Weak, advisory signal.** Handwriting varies; a model's visual opinion is not identity matching. Copy must never claim verification.
- **Reviewer-initiated.** With no reference, no comparison happens. A reference is per case; the library is data collection only.
- **Detection depends on a vision model** and, for non-embedded marks, on its localisation.
- The reuse check needs a **text layer** to compare signer names (scans fall back to `identical_reuse`), and only sees marks that were cropped in the first place.
- The reference's kind is inferred from overlap; a box that overlaps neither region leaves it unknown (and skips the reuse check).
- The task and endpoints are PDF-only.

Risk rules fed

`rule_id`	Condition	Weight	Severity
`signature.expected_missing`	detection `flag` (expected but none located)	10	low
`signature.inconsistent_with_reference`	verdict `inconsistent`	25	medium
`signature.possibly_consistent_only`	verdict `possibly_consistent`	5	low
`signature.identical_reuse`	verdict `identical_reuse`	15	medium
`signature.reused_different_signer`	verdict `reused_different_signer`	35	high

`cannot_determine` and `consistent` fire no default rule (a company's Reviewer L2 can add one for `cannot_determine`).